FastAPI Reference

`UploadFile` class

You can define *path operation function* parameters to be of the type `UploadFile` to receive files from the request.

You can import it directly from `fastapi`:

```
from fastapi import UploadFile
```

`class` fastapi.UploadFile

```
UploadFile(file, *, size=None, filename=None, headers=None)
```

Bases: `UploadFile`

A file uploaded in a request.

Define it as a *path operation function* (or dependency) parameter.

If you are using a regular `def` function, you can use the `upload_file.file` attribute to access the raw standard Python file (blocking, not async), useful and needed for non-async code.

Read more about it in the FastAPI docs for Request Files.

Example

```
from typing import Annotated

from fastapi import FastAPI, File, UploadFile

app = FastAPI()


@app.post("/files/")
async def create_file(file: Annotated[bytes, File()]):
    return {"file_size": len(file)}


@app.post("/uploadfile/")
async def create_upload_file(file: UploadFile):
    return {"filename": file.filename}
```

Ask AI

Source code in `starlette/datastructures.py`

```
def __init__(
    self,
    file: BinaryIO,
    *,
    size: int | None = None,
    filename: str | None = None,
    headers: Headers | None = None,
) -> None:
    self.filename = filename
    self.file = file
    self.size = size
    self.headers = headers or Headers()

    # Capture max size from SpooledTemporaryFile if one is provided. This slightly speeds up
future checks.
    # Note 0 means unlimited mirroring SpooledTemporaryFile's __init__
    self._max_mem_size = getattr(self.file, "_max_size", 0)
```

attr file `instance-attribute`

```
file
```

The standard Python file object (non-async).

attr filename `instance-attribute`

```
filename
```

The original file name.

attr size `instance-attribute`

```
size
```

The size of the file in bytes.

attr headers `instance-attribute`

```
headers
```

The headers of the request.

attr content_type `instance-attribute`

```
content_type
```

The content type of the request, from the headers.

meth read `async`

```
read(size=-1)
```

Read some bytes from the file.

To be awaitable, compatible with async, this is run in threadpool.

PARAMETER	DESCRIPTION
`size`	The number of bytes to read from the file. **TYPE:** `int` **DEFAULT:** `-1`

Source code in `fastapi/datastructures.py`

```
async def read(
    self,
    size: Annotated[
        int,
        Doc(
            """
            The number of bytes to read from the file.
            """
        ),
    ] = -1,
) -> bytes:
    """
    Read some bytes from the file.

    To be awaitable, compatible with async, this is run in threadpool.
    """
    return await super().read(size)
```

`meth` write `async`

```
write(data)
```

Write some bytes to the file.

You normally wouldn't use this from a file you read in a request.

To be awaitable, compatible with async, this is run in threadpool.

PARAMETER	DESCRIPTION
`data`	The bytes to write to the file. **TYPE:** `bytes`

Source code in `fastapi/datastructures.py`

```
async def write(
    self,
    data: Annotated[
        bytes,
        Doc(
            """
            The bytes to write to the file.
            """
        ),
    ],
) -> None:
    """
    Write some bytes to the file.

    You normally wouldn't use this from a file you read in a request.

    To be awaitable, compatible with async, this is run in threadpool.
    """
    return await super().write(data)
```

`meth` seek `async`

```
seek(offset)
```

Move to a position in the file.

Any next read or write will be done from that position.

To be awaitable, compatible with async, this is run in threadpool.

PARAMETER	DESCRIPTION
`offset`	The position in bytes to seek to in the file. **TYPE:** `int`

Source code in `fastapi/datastructures.py`

```
async def seek(
    self,
    offset: Annotated[
        int,
        Doc(
            """
            The position in bytes to seek to in the file.
            """
        ),
    ],
) -> None:
    """
    Move to a position in the file.

    Any next read or write will be done from that position.

    To be awaitable, compatible with async, this is run in threadpool.
    """
    return await super().seek(offset)
```

`meth` close `async`

```
close()
```

Close the file.

To be awaitable, compatible with async, this is run in threadpool.

Source code in `fastapi/datastructures.py`

```
124 async def close(self) -> None:
125     """
126     Close the file.
127 
128     To be awaitable, compatible with async, this is run in threadpool.
129     """
130     return await super().close()
```